

Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 «Программная инженерия» –

Системное и прикладное программное обеспечение

Дисциплина: Рефакторинг баз данных и приложений

План рефакторинга

Выполнили:

студенты 4 курса

Кравцов Кирилл Денисович

Группа: Р3411

Батманов Даниил Евгеньевич

Группа: Р3407

Преподаватель:

Логинов Иван Павлович

г. Санкт-Петербург, 2025

Ссылка на GitHub репозиторий

Общая информация по проекту

Описание проекта

Информационная система для управления космической станцией, специализирующейся на производстве, хранении и торговле ресурсами. Включает модули разных типов, управление пользователями, кораблями, производством и торговыми операциями.

Функциональность проекта

- Управление пользователями (4 роли: Владелец, Менеджер, Инженер, Пилот)
- Регистрация кораблей и стыковка к станции
- Строительство модулей (производственные, складские, стыковочные)
- Производство ресурсов (циклы с потреблением/выпуском)
- Торговля ресурсами с гибкими политиками
- Управление финансами и складами
- JWT-аутентификация и ролевой доступ

Технологический стек

Backend: Spring Boot, Security, Data JPA, Web

База данных: PostgreSQL, Flyway (миграции)

Документация: Swagger

Сборка: Maven

Контейнеризация: Docker

Итерации рефакторинга

Этап 1: Инфраструктура и документация

1.1. Вынесение OpenAPI из кода

- *Проблема:* Документация API генерируется только при запущенном приложении
- *Задачи:*
 - Создать отдельную спецификацию OpenAPI в формате YAML
- *Цель:* Независимая от кода актуальная документация API

1.2. Вынесение миграций из приложения

- *Проблема:* Миграции БД выполняются при старте приложения
- *Задачи:*
 - Вынести Flyway миграции в отдельный процесс
 - Настроить запуск миграций до старта приложения
- *Цель:* Разделение ответственности и контроль над миграциями

1.3. Миграция системы сборки на Gradle

- *Проблема:* Использование устаревшей системы сборки Maven
- *Задачи:*
 - Конвертировать pom.xml в build.gradle.kts
 - Настроить кэширование и инкрементальную сборку
 - Оптимизировать зависимости
- *Цель:* Современная и производительная система сборки

Этап 2: Миграция на Kotlin и модернизация

2.1. Переписывание кода с Java на Kotlin

- *Проблема:* Устаревший Java-код с boilerplate
- *Задачи:*
 - Постепенная миграция классов на Kotlin
 - Использование Kotlin features (data classes, extension functions)
 - Рефакторинг с учетом null-safety
- *Цель:* Современный, безопасный и лаконичный код

2.2. Внедрение типобезопасной конфигурации

- *Проблема:* Рассыпанные по коду @Value аннотации
- *Задачи:*
 - Создать @ConfigurationProperties классы
 - Валидация конфигурации при старте
 - Группировка связанных настроек
- *Цель:* Централизованная и безопасная конфигурация

2.3. Внедрение структурного логирования

- *Проблема:* Неструктурированные логи с конкатенацией строк
- *Задачи:*
 - JSON-форматирование логов для ELK
 - Унификация формата лог-сообщений
- *Цель:* Машинно-читаемые логи для мониторинга

2.4. Обновление зависимостей

- *Проблема:* Устаревшие версии Spring Boot и других библиотек
- *Задачи:*
 - Обновление Spring Boot до 4.x

- Обновление остальных зависимостей
- Тестирование на совместимость
- *Цель:* Актуальные и безопасные зависимости

Этап 3: Модульность и контейнеризация

3.1. Разделение на модули с Spring Modulith

- *Проблема:* Монолитная структура приложения
- *Задачи:*
 - Выделение модулей (users, trading, production, docking)
 - Настройка межмодульного взаимодействия
 - Верификация архитектурных правил
- *Цель:* Четкое разделение ответственности

3.2. Оптимизация конфигурации Docker

- *Проблема:* Сложный процесс локальной сборки и запуска, требующий установки дополнительных инструментов (Java, Gradle), что замедляет onboarding и усложняет демонстрацию проекта.
- *Задачи:*
 - Обеспечить возможность запуска всего стека одной командой `docker-compose up`
 - Устранить необходимость локальной установки Java/Kotlin/Gradle
 - Упростить onboarding новых разработчиков
 - Обеспечить единообразие окружений
- *Цель:* Создать оптимизированную Docker-конфигурацию с multi-stage Dockerfile и docker-compose для запуска полного стека (БД, миграции, приложение).

3.3. Актуализация документации после рефакторинга

- *Проблема:* После проведения рефакторинга документация устаревает и не отражает текущее состояние проекта.
- *Задачи:*
 - Обеспечить соответствие документации реальному состоянию кода
 - Документировать новую модульную архитектуру
 - Обновить инструкции по разработке и развертыванию
 - Описать изменения в конфигурации и зависимостях
- *Цель:* Обновить всю проектную документацию (README, инструкции, API документация) в соответствии с изменениями, внесенными в ходе рефакторинга.